

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

Course Code: CSE 219 Course Title: Signals and Linear Systems

Time: 25 minutes

Marks: 20

Student Name: Md. Ashrafur Rahman Khan

Student ID: 1905005

Answer **exactly 2 out of Questions 1–4. Question 5 is mandatory.** Answer concisely.

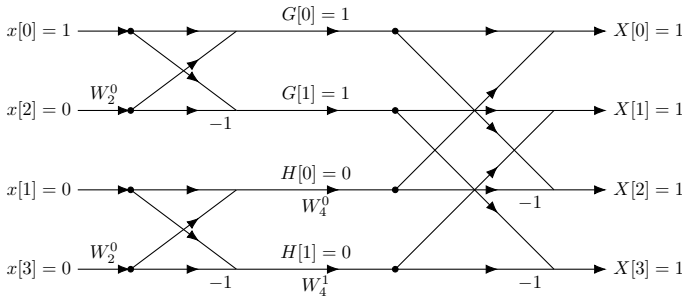
1. Consider the signal $x[n]$ defined as follows for $N = 4$:

[8]

$$x[n] = \begin{cases} 1, & \text{if } n = 0 \\ 0, & \text{if } 1 \leq n < N \end{cases}$$

- (a) Compute the 4-point radix-2 DIT FFT of the signal. Draw a butterfly diagram and show your calculations on the diagram.

Solution: The sequence is $x[n] = \{1, 0, 0, 0\}$. For the 4-point radix-2 DIT FFT, the inputs are arranged in bit-reversed order: $x[0], x[2], x[1], x[3]$, which corresponds to 1, 0, 0, 0.



- (b) If $x[n]$ is viewed as samples of an aperiodic continuous-time signal, what might be a continuous-time signal corresponding to it?

Solution: A continuous-time signal corresponding to it could be the Dirac delta function $\delta(t)$ or a very narrow pulse at $t = 0$, since the sequence only has a single non-zero value at $n = 0$.

- (c) Based on your answer in (b), interpret your result from (a). If the sequence is zero-padded to length 32, what is its 32-point DFT? Briefly justify.

Solution: The continuous-time Fourier transform of $\delta(t)$ is 1 for all frequencies. Similarly, the DTFT of the single impulse $\delta[n]$ is $X(e^{j\omega}) = 1$ for all ω . The N -point DFT evaluates $X(e^{j\omega})$ exactly at evenly spaced discrete frequencies. Thus, it is simply N equispaced samples of the continuous DTFT exactly over one 2π period. This gives $X[k] = 1$ for all k , which matches our result from (a).

By zero-padding the sequence to a total length $M = 32$, taking the 32-point DFT samples the *same* continuous DTFT spectrum $X(e^{j\omega})$, but at a much finer resolution of $2\pi/32$. Since the underlying DTFT is constantly 1, the 32-point DFT $X_{32}[k]$ will just be 1 for all $k = 0, \dots, 31$.

- (d) In a radix-2 DIT FFT, we typically use bit-reversed order for the input $x[n]$. Imagine you forgot to use bit-reversed order and used natural order for the input instead, resulting in a sequence $X'[k]$. Determine the signal $x'[n]$ whose DFT is actually $X'[k]$.

Solution: The FFT algorithm expects the input at algorithmic index n to be $x[r(n)]$, where $r(n)$ is the bit-reversed value of n . So if we provide $x[n]$ directly, the algorithm mathematically processes it as if it were the bit-reversed version of some actual virtual signal $x'[n]$. Thus, $x'[n] = x[r(n)]$. The signal $x'[n]$ is simply the bit-reversed sequence of $x[n]$.

In this specific case, $x[n] = \{1, 0, 0, 0\}$ for $n = 0, 1, 2, 3$. The bit-reversed indices process as follows: $x'[0] = x[0] = 1$, $x'[1] = x[2] = 0$, $x'[2] = x[1] = 0$, $x'[3] = x[3] = 0$. Thus, $x'[n] = \{1, 0, 0, 0\}$. Since the only non-zero position is at index 0 and remains unchanged, $x'[n] = x[n]$.

2. In many cases, the radix-4 FFT is highly efficient. Suppose you are deriving the radix-4 DIT Cooley-Tukey algorithm. You have already established that for $0 \leq k < N/4$:

[8]

$$X[k] = G_0[k] + W_N^k G_1[k] + W_N^{2k} G_2[k] + W_N^{3k} G_3[k]$$

Where

$$G_m[k] = \sum_{r=0}^{N/4-1} x[4r+m] W_{N/4}^{kr}, \quad m = 0, 1, 2, 3$$

- (a) Derive the formulas for $X[k + N/4]$, $X[k + 2N/4]$, and $X[k + 3N/4]$.

Solution: Since $G_m[k]$ is a DFT of length $N/4$, it is periodic with period $N/4$, so $G_m[k + pN/4] = G_m[k]$. Also, note the twiddle factor phase shifts: $W_N^{pN/4} = e^{-j\frac{2\pi}{N} \frac{pN}{4}} = e^{-j\frac{\pi}{2}p} = (-j)^p$. Using these properties:

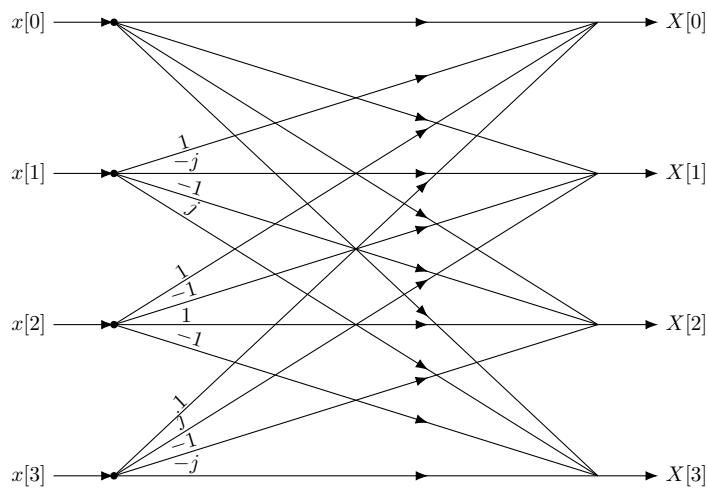
$$\begin{aligned} X[k + N/4] &= G_0[k] + W_N^{k+N/4} G_1[k] + W_N^{2(k+N/4)} G_2[k] + W_N^{3(k+N/4)} G_3[k] \\ &= G_0[k] - jW_N^k G_1[k] - W_N^{2k} G_2[k] + jW_N^{3k} G_3[k] \\ X[k + 2N/4] &= G_0[k] + W_N^{k+2N/4} G_1[k] + W_N^{2(k+2N/4)} G_2[k] + W_N^{3(k+2N/4)} G_3[k] \\ &= G_0[k] - W_N^k G_1[k] + W_N^{2k} G_2[k] - W_N^{3k} G_3[k] \\ X[k + 3N/4] &= G_0[k] + W_N^{k+3N/4} G_1[k] + W_N^{2(k+3N/4)} G_2[k] + W_N^{3(k+3N/4)} G_3[k] \\ &= G_0[k] + jW_N^k G_1[k] - W_N^{2k} G_2[k] - jW_N^{3k} G_3[k] \end{aligned}$$

- (b) Draw the radix-4 butterfly for the case $N = 4$.

Hint: Instead of just ± 1 , there will also be multiplications involving $\pm j$.

Solution: For $N = 4$, $N/4 = 1$, so $G_m[0] = x[m]$. There is only $k = 0$ to evaluate for each branch. The equations simplify to:

$$\begin{aligned} X[0] &= x[0] + x[1] + x[2] + x[3] \\ X[1] &= x[0] - jx[1] - x[2] + jx[3] \\ X[2] &= x[0] - x[1] + x[2] - x[3] \\ X[3] &= x[0] + jx[1] - x[2] - jx[3] \end{aligned}$$



3. Consider two polynomials:

[8]

$$A(x) = \sum_{n=0}^{N-1} a[n]x^n \quad \text{and} \quad B(x) = \sum_{n=0}^{N-1} b[n]x^n$$

You want to compute their product, $C(x) = A(x)B(x)$.

- (a) Provide a simple, naive algorithm to find the polynomial $C(x)$ and analyze its time complexity.

Solution: A naive algorithm expands the product algebraically.

```
1 function NaivePolynomialMult(a, b, N):
2     // a and b are arrays of size N
3     c = array of size 2N-1 initialized to 0
4     for i from 0 to N-1:
5         for j from 0 to N-1:
6             c[i + j] = c[i + j] + a[i] * b[j]
7     return c
```

Because there are two nested loops of size N , the time complexity is $O(N^2)$.

- (b) If we express $C(x)$ as the sum $\sum c[n]x^n$, determine the valid range for n and provide a general formula for the coefficients $c[n]$.

Solution: The maximum degree in $C(x)$ is $(N - 1) + (N - 1) = 2N - 2$. The minimum degree is 0. Therefore, the valid range for n is $0 \leq n \leq 2N - 2$.

First, the standard convolution sum is $c[n] = \sum_{k=0}^{N-1} a[k]b[n - k]$, assuming a and b are implicitly zero-padded outside the range $[0, N - 1]$.

To find the exact non-zero bounds for a specific n : we know $a[k]$ is defined for $0 \leq k \leq N - 1$. Similarly, $b[n - k]$ is defined for $0 \leq n - k \leq N - 1$. From $n - k \geq 0$, we get $k \leq n$. Combining with $k \leq N - 1$ gives the upper bound: $k \leq \min(n, N - 1)$. From $n - k \leq N - 1$, we get $k \geq n - N + 1$. Combining with $k \geq 0$ gives the lower bound: $k \geq \max(0, n - N + 1)$. Applying these bounds results in the highly specific

general formula:

$$c[n] = \sum_{k=\max(0, n-N+1)}^{\min(n, N-1)} a[k]b[n-k]$$

- (c) Express the coefficient sequence $c[n]$ as a convolution.

Solution: The coefficient sequence $c[n]$ is exactly the linear convolution of the sequences $a[n]$ and $b[n]$:

$$c[n] = (a * b)[n] = \sum_{k=-\infty}^{\infty} a[k]b[n-k]$$

where $a[n]$ and $b[n]$ are assumed to be zero outside the range $0 \leq n \leq N-1$.

- (d) Propose a more efficient algorithm to compute $C(x)$ and analyze the time complexity of your proposed algorithm.

Solution: An efficient algorithm uses the Fast Fourier Transform (FFT) to perform fast linear convolution:

```

1 function FastPolynomialMult(a, b, N):
2     // Find next power of 2 >= 2N - 1
3     M = NextPowerOfTwo(2N - 1)
4     // Zero-pad to length M
5     a_padded = ZeroPad(a, M)
6     b_padded = ZeroPad(b, M)
7     // Transform to frequency domain
8     A = FFT(a_padded)
9     B = FFT(b_padded)
10    // Pointwise multiplication
11    for k from 0 to M-1:
12        C[k] = A[k] * B[k]
13    // Inverse transform
14    c = IFFT(C)
15    // Return the First 2N-1 coefficients
16    return c[0 ... 2N-2]
```

Complexity: The zero-padding takes $O(N)$ time. The two forward FFTs and one inverse FFT of length $M \approx 2N$ each take $O(M \log M) = O(N \log N)$ running time. The pointwise multiplication takes $O(M) = O(N)$ time. Overall time complexity is $O(N \log N)$, which is significantly faster than the naive $O(N^2)$ algorithm.

4. In image processing, we often use a 2-dimensional version of the DFT, which is defined as:

[8]

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}$$

Where $x[n_1, n_2]$ is an $N_1 \times N_2$ matrix and $W_N^k = e^{-j2\pi k/N}$.

- (a) What is the time complexity of a naive algorithm used to compute the 2D DFT?

Solution: For each of the $N_1 N_2$ frequency pairs (k_1, k_2) , the naive algorithm evaluates a double sum involving $N_1 N_2$ terms:

```

1 function Naive2D_DFT(x, N1, N2):
2     X = matrix of size N1 x N2 initialized to 0
3     for k1 from 0 to N1-1:
4         for k2 from 0 to N2-1:
5             for n1 from 0 to N1-1:
6                 for n2 from 0 to N2-1:
7                     X[k1, k2] += x[n1, n2]*exp(-j*2*pi*(k1*n1/N1 + k2*n2/N2))
8     return X
```

Thus, it takes $O(N_1 N_2)$ operations per frequency, leading to an overall time complexity of $O((N_1 N_2)^2) = O(N_1^2 N_2^2)$.

- (b) How can you interpret the given formula as two successive 1D DFT operations?

Solution: We can separate the summations as follows:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \underbrace{\left(\sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{k_2 n_2} \right)}_{\text{1D DFT along rows}} W_{N_1}^{k_1 n_1}$$

The inner sum is simply a 1D DFT of length N_2 performed along each row n_1 of the matrix. Let this

result be a temporary matrix $X'[n_1, k_2]$. Then, the outer sum computes a 1D DFT of length N_1 down each column k_2 of the temporary matrix X' .

- (c) Propose an efficient algorithm (2D FFT) for computing this and analyze its complexity.

Solution: Using the interpretation from part (b), we can compute the 2D DFT efficiently by leveraging 1D FFT algorithms (known as the row-column algorithm):

```
1 function Fast2D_DFT(x, N1, N2):
2     X_temp = matrix of size N1 x N2
3     // 1D FFT along rows
4     for n1 from 0 to N1-1:
5         X_temp[n1, :] = FFT_1D(x[n1, :], N2)
6     X = matrix of size N1 x N2
7     // 1D FFT down columns
8     for k2 from 0 to N2-1:
9         X[:, k2] = FFT_1D(X_temp[:, k2], N1)
10    return X
```

Complexity Analysis: The first step requires N_1 independent FFTs of length N_2 , taking $O(N_1 \cdot N_2 \log N_2)$ total operations. The second step requires N_2 independent FFTs of length N_1 , taking $O(N_2 \cdot N_1 \log N_1)$ total operations. Adding them together yields $O(N_1 N_2 \log N_2 + N_1 N_2 \log N_1) = O(N_1 N_2 \log(N_1 N_2))$. This is much faster than the naive $O(N_1^2 N_2^2)$.

- (d) What is the primary difference between your 2D FFT algorithm and Bailey's FFT algorithm?

Solution: While both algorithms use row and column 1D FFTs on a 2D matrix structure, they solve fundamentally different problems. Bailey's FFT algorithm computes a **1D FFT** of length $N = N_1 N_2$ by mapping a 1D sequence into a 2D matrix. Critically, it requires intermediate multiplications by **twiddle factors** ($W_N^{n_1 k_2}$) after the row FFTs and before performing the column FFTs. The 2D FFT algorithm computes the **2D DFT** of a naturally 2D signal and involves **no internal twiddle factors** between the sequence of row and column FFTs.

5. Thank you for attending the classes and participating in this CT.

[4]

Solution: You're welcome!